# Ch9 進階字串處理

# 9-1 ASCII 編碼

- ASCII 編碼是最早的電腦編碼系統，只包含大小寫英文字母、數字、英文標點符號、數學運算符號與控制符號等，共128 個字。
- 將二進位編碼的控制符號換算成十進位，控制符號編碼介於0 到31 共32 個，再加上127，所以控制符號共有33 個字

# 9-1-1　模組string ( ch9\9-1-1- 模組string.py)

- 模組string 提供ASCII 字碼，相似功能的字元分在一起，如以下分類。

表 9-1　模組 string 說明

| string 模組分類名稱 | 說明 | 結果 | 元素個數 |
|---|---|---|---|
| string.ascii_letters | 大小寫英文字母 | abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ | 52 |
| string.ascii_lowercase | 小寫英文字母 | abcdefghijklmnopqrstuvwxyz | 26 |
| string.ascii_uppercase | 大寫英文字母 | ABCDEFGHIJKLMNOPQRSTUVWXYZ | 26 |
| string.digits | 十進位數字 | 0123456789 | 10 |
| string.hexdigits | 十六進位數字 | 0123456789abcdefABCDEF | 22 |
| string.octdigits | 八進位數字 | 01234567 | 8 |
| string.punctuation | 標點符號 | !”#$%&'()*+,-./:;<=>?@[\]^_`{\|}~ | 32 |

# 9-1-1 模組string (ch9\9-1-1- 模組string.py)

| | | | |
|---|---|---|---|
| string.printable | 包含十進位數字、大小寫英文字母、標點符號與空白字符。 | 0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!” #$%&'()*+,-./:;<=>?@[\]^_`{\|}~ 0x20(space)、0x09(tab,\t)、0x0a(line feed,\n)、0x0b(vertical tab,\v)、0x0c(form feed)、0x0d(carriage return,\r) | 100 |
| string.whitespace | 空白字符 | 0x20(space)、0x09(tab,\t)、0x0a(line feed,\n)、0x0b(vertical tab,\v)、0x0c(form feed)、0x0d(carriage return,\r) | 6 |

# 9-1-1 模組string ( ch9\9-1-1- 模組string.py)

| 行號 | 範例 ( : ch9\9-1-1- 模組 string.py) |
|---|---|
| 1 | import string |
| 2 | print('ascii_letters 爲 ', string.ascii_letters) |
| 3 | print('ascii_lowercase 爲 ', string.ascii_lowercase) |
| 4 | print('ascii_uppercase 爲 ', string.ascii_uppercase) |
| 5 | print('digits 爲 ', string.digits) |
| 6 | print('hexdigits 爲 ', string.hexdigits) |
| 7 | print('octdigits 爲 ', string.octdigits) |
| 8 | print('punctuation 爲 ', string.punctuation) |
| 9 | print("printable.encode('ascii') 爲 ", string.printable.encode('ascii')) |
| 10 | print("whitespace.encode('ascii') 爲 ", string.whitespace.encode('ascii')) |

# 程式解說

- 第1 行：匯入模組string。
- 第2 行：顯示字串「ascii_letters為」與模組string 的字串ascii_letters到螢幕上。
- 第3行：顯示字串「ascii_lowercase為」與模組string的字串ascii_lowercase到螢幕上。
- 第4行：顯示字串「ascii_uppercase為」與模組string的字串ascii_uppercase到螢幕上。

# 程式解說

- 第5 行：顯示字串「digits為」與模組string 的字串digits到螢幕上。
- 第6 行：顯示字串「hexdigits為」與模組string 的字串hexdigits到螢幕上。
- 第7 行：顯示字串「octdigits為」與模組string 的字串octdigits到螢幕上。
- 第8 行：顯示字串「punctuation為」與模組string 的字串punctuation到螢幕上。

# 程式解說

- 第9 行：顯示字串「printable.encode('ascii') 為」與模組string 的字串printable 到螢幕上，因為printable 最後6 個空白字元只有space 可以顯示，其餘無法顯示
- 所以使用函式encode，將ascii 字元轉換成byte 的十六進位表示，最後5 個空白字元轉換成「\t\n\r\x0b\x0c」顯示在螢幕上。

- 第10 行：顯示字串「whitespace.encode('ascii') 為」與模組string 的字串whitespace到螢幕上，最後6個空白字元只有space 可以顯示，其餘無法顯示
- 所以使用函式encode，將ascii 字元轉換成byte 的十六進位表示，最後5 個空白字元轉換成「\t\n\r\x0b\x0c」顯示在螢幕上。

# 9-1-1 模組string ( ch9\9-1-1- 模組string.py)

## 執行結果

```
ascii_letters 爲 abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
ascii_lowercase 爲 abcdefghijklmnopqrstuvwxyz
ascii_uppercase 爲 ABCDEFGHIJKLMNOPQRSTUVWXYZ
digits 爲 0123456789
hexdigits 爲 0123456789abcdefABCDEF
octdigits 爲 01234567
punctuation 爲 !"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
printable.encode('ascii') 爲 b'0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKL
MNOPQRSTUVWXYZ!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~ \t\n\r\x0b\x0c'
whitespace.encode('ascii') 爲 b' \t\n\r\x0b\x0c'
```

# 9-1-2　密碼隨機產生器 (ch9\9-1-2- 密碼隨機產生器.py)

- 隨機產生長度為8 到12 的密碼，密碼由大小寫英文字母與數字所組成。
  - 提示：使用random.randint(8,12) 隨機產生8到12 其中之一的數字，控制for 迴圈產生長度為8 到12 的密碼。

# 9-1-2　密碼隨機產生器 (ch9\9-1-2- 密碼隨機產生器.py)

| 行號 | 範例 (  : ch9\9-1-2- 密碼隨機產生器 .py) | 執行結果 |
|---|---|---|
| 1 | import string | |
| 2 | import random | |
| 3 | chs = string.ascii_letters + string.digits | |
| 4 | pwd="" | vEaMlDaG |
| 5 | for x in range(random.randint(8,12)): | |
| 6 | 　pwd+=random.choice(chs) | |
| 7 | print(pwd) | |

# 程式解說

- 第1 行：匯入模組string。
- 第2 行：匯入模組random。
- 第3行：設定chs為大小寫英文字母(string.ascii_letters)串接數字0到9(string.digits)。

- 第4 行：設定pwd 為空字串。
- 第5 到6 行：使用random.randint(8,12) 會回傳8 到12 的數字，使用for 迴圈與range(random.randint(8,12)) 控制迴圈執行8 到12 次，每次隨機從字串chs 選出一個字元串接到pwd。
- 第7 行：顯示pwd 到螢幕上。

# 9-2 Unicode 編碼

- Unicode 編碼整合全世界各個國家的文字編碼，讓Unicode 編碼可以通行全世界，支援多語言環境，不用每個國家都自訂一種編碼，讓資訊系統可以更簡單進行文字編碼與儲存，Python3 的預設文字編碼為 Unicode 編碼。

# 9-2-1　Unicode 字元的表示 ( ch9\9-2-1-Unicode 字元.py)

- 一個Unicode 字元可以經由十六進位編碼表示，也可以使用標準名稱表示，其中十六進位編碼又可以分成4 個十六進位表示或8 個十六進位表示，所以可以區分成以下三種。

- 1. 使用「\u」表示使用4 個十六進位表示，例如：「\u6211」表示「我」的Unicode 編碼。
- 2. 使用「\U」表示使用8 個十六進位表示，例如：「\U00006211」表示「我」的Unicode 編碼。
- 3. 使用「\N」表示使用標準名稱，例如：「\N{CJK UNIFIED IDEOGRAPH-6211}」表示「我」的Unicode 標準名稱。

# 9-2-1 Unicode 字元的表示 ( ch9\9-2-1-Unicode 字元.py)

- 使用模組unicodedata 中的函式name，將Unicode 字元轉換成Unicode 標準名稱
- 使用函式lookup 將Unicode 標準名稱轉換成Unicode 字元，以下程式碼，使用函式name 找出「我」的Unicode 標準名稱。
  - import unicodedata
  - print(unicodedata.name(' 我'))

- 顯示「我」的Unicode 標準名稱如下。
  - CJK UNIFIED IDEOGRAPH-6211

# 9-2-1 Unicode 字元的表示 ( ch9\9-2-1-Unicode 字元.py)

- 以下程式碼， 使用函式lookup 找出Unicode 標準名稱「CJK UNIFIED IDEOGRAPH-6211」的對應的Unicode 字元。
  - import unicodedata
  - print(unicodedata.lookup('CJK UNIFIED IDEOGRAPH-6211'))

- 找出Unicode 標準名稱「CJK UNIFIED IDEOGRAPH-6211」的對應的Unicode 字元，如下：
  - 我

- 「我」也可以使用「\u6211」或「\U00006211」或「\N{CJK UNIFIED IDEOGRAPH-6211}」取代。
  - import unicodedata
  - print(unicodedata.name('\u6211'))
  - print(unicodedata.name('\U00006211'))
  - print(unicodedata.name('\N{CJK UNIFIED IDEOGRAPH-6211}'))

- 顯示「\u6211」、「\U00006211」與「\N{CJK UNIFIED IDEOGRAPH-6211}」的Unicode 標準名稱如下。
  - CJK UNIFIED IDEOGRAPH-6211
  - CJK UNIFIED IDEOGRAPH-6211
  - CJK UNIFIED IDEOGRAPH-6211

# 9-2-1 Unicode 字元的表示 ( ch9\9-2-1-Unicode 字元.py)

- 綜合上述概念，自訂函式unicode_name將Unicode 字元轉換成Unicode 標準名稱，並顯示Unicode 字元與Unicode 標準名稱，並回傳Unicode 標準名稱，

- 自訂函式unicode_lookup 將Unicode 標準名稱轉換成Unicode 字元，並顯示Unicode 字元與Unicode 標準名稱，並回傳Unicode 字元。

| 行號 | 範例 (  ：ch9\9-2-1-Unicode 字元 .py) |
|---|---|
| 1 | import unicodedata |
| 2 | def unicode_name(value): |
| 3 | name = unicodedata.name(value) |
| 4 | print('value=', value, 'name=', name) |
| 5 | return name |
| 6 | def unicode_lookup(name): |
| 7 | value = unicodedata.lookup(name) |
| 8 | print('name=', name, 'value=', value) |
| 9 | return value |
| 10 | name = unicode_name(' 我 ') |
| 11 | value = unicode_lookup(name) |
| 12 | name = unicode_name('\u6211') |
| 13 | value = unicode_lookup(name) |
| 14 | name = unicode_name('\U00006211') |
| 15 | value = unicode_lookup(name) |
| 16 | name = unicode_name('\N{CJK UNIFIED IDEOGRAPH-6211}') |
| 17 | value = unicode_lookup(name) |

# 程式解說

- 第1 行：匯入模組unicodedata。
- 第2 到5 行：定義函式unicode_name，輸入Unicode 字元參數value，呼叫模組unicodedata 的函式name，以value 為輸入，變數name參考到要回傳的Unicode標準名稱（第3 行）
- 使用函式print 顯示Unicode 字元與Unicode 標準名稱到螢幕上（第4 行），最後回傳name（第5 行）。

- 第6 到9 行：定義函式unicode_lookup，輸入Unicode 標準名稱參數name，呼叫模組unicodedata 的函式lookup，以name為輸入，變數value參考到要回傳的Unicode字元（第7 行）
- 使用函式print 顯示Unicode 標準名稱與Unicode 字元到螢幕上（第8 行），最後回傳value（第9 行）。

- 第10 行：呼叫函式unicode_name，以「我」為輸入參數，變數name參考到函式回傳結果。
- 第11 行：呼叫函式unicode_lookup，以name 為輸入參數，變數value參考到函式回傳結果。

- 第12 行：呼叫函式unicode_name，以「\u6211」為輸入參數，變數name參考到函式回傳結果。
- 第13 行：呼叫函式unicode_lookup，以name 為輸入參數，變數value參考到函式回傳結果。
- 第14 行：呼叫函式unicode_name，以「\U00006211」為輸入參數，變數name參考到函式回傳結果。

- 第15 行：呼叫函式unicode_lookup，以name 為輸入參數，變數value參考到函式回傳結果。
- 第16 行：呼叫函式unicode_name，以「\N{CJK UNIFIED IDEOGRAPH-6211}」為輸入參數，變數name參考到函式回傳結果。
- 第17 行：呼叫函式unicode_lookup，以name 為輸入參數，變數value參考到函式回傳結果。

# 9-2-1 Unicode 字元的表示 ( ch9\9-2-1-Unicode 字元.py)

## 執行結果

```
value= 我 name= CJK UNIFIED IDEOGRAPH-6211
name= CJK UNIFIED IDEOGRAPH-6211 value= 我
value= 我 name= CJK UNIFIED IDEOGRAPH-6211
name= CJK UNIFIED IDEOGRAPH-6211 value= 我
value= 我 name= CJK UNIFIED IDEOGRAPH-6211
name= CJK UNIFIED IDEOGRAPH-6211 value= 我
value= 我 name= CJK UNIFIED IDEOGRAPH-6211
name= CJK UNIFIED IDEOGRAPH-6211 value= 我
```

# 9-2-2　編碼與解碼 (ch9\9-2-2- 編碼與解碼.py)

- 編碼(encode)
  - 將字串轉換成位元組(byte) 稱作編碼(encode)，而將已編碼的位元組(byte) 還原回原來的字串稱作解碼(decode)

- 編碼(encode)
  - 現在存在許多編碼系統，目前最常用的是UTF-8 編碼，由Unicode 編碼轉換而來，讓每個Unicode 字元由1 到4 個位元組表示，原來ASCII 使用1 個位元組表示，拉丁文、希臘文、阿拉伯文…等使用2 個位元組表示

- 編碼(encode)
  - 大部分的中文使用3 個位元組表示，古義大利字母(Old Italic)、日文假名補充(KanaSupplement)、音樂符號(Musical Symbols)…等使用4 個位元組表示。

# 9-2-2　編碼與解碼 (ch9\9-2-2- 編碼與解碼.py)

表 9-2　函式 encode 所支援的常見編碼

| 編碼代碼 | 說明 | 使用方式 |
|---|---|---|
| ascii | 7 位元的 ASCII。 | 字串 .encode('ascii') |
| utf-8 | 將 Unicode 編碼轉換成可變長度的編碼，以 1 到 4 個位元組表示一個 Unicode 字元。 | 字串 .encode('utf-8') |

| 編碼代碼 | 說明 | 使用方式 |
|---|---|---|
| unicode-escape | 將 Unicode 編碼使用「\u」與「\U」表示。 | 字串 .encode('unicode-escape') |
| latin-1 | 也就是 ISO 8859-1，以 ascii 爲基礎，支援使用於歐洲的語言。 | 字串 .encode('latin-1') |

# 9-2-2 編碼與解碼 (ch9\9-2-2- 編碼與解碼.py)

- 任何字串加上「.encode('unicode-escape')」，就可以轉換成Unicode 編碼，例如要找出「我」的Unicode 編碼，程式碼如下。
  - print(' 我'.encode('unicode-escape'))
- 輸出「我」的Unicode 編碼如下。
  - b'\\u6211'

# 9-2-2　編碼與解碼 (ch9\9-2-2- 編碼與解碼.py)

- 解碼(decode)
  - Python 提供函式decode 可以將十六進位表示的位元組解碼回原來的字串，也需指定解碼代碼，解碼代碼與編碼代碼要相同，編碼時指定的編碼代碼，解碼時也要使用相同的代碼進行解碼，解碼才會正確回到編碼前的字串。

# 9-2-2 編碼與解碼 (ch9\9-2-2- 編碼與解碼.py)

- 將「我」進行Unicode 編碼與解碼程式碼如下。

```
byte = '我'.encode('unicode-escape')
s = byte.decode('unicode-escape')
print(byte, s)
```

程式執行結果如下。

```
b'\\u6211' 我
```

# 9-2-2　編碼與解碼 (ch9\9-2-2- 編碼與解碼.py)

- 將「我」使用utf-8 進行編碼結果為「\xe6\x88\x91」，使用函式len 測量字串「我」的長度會發現長度為1，使用utf-8 進行編碼後為「\xe6\x88\x91」，長度為3，編碼後字串長度以byte 計算，所以長度變長

- 使用函式len 計算字串長度的程式碼如下。
  - print(len(' 我'))
  - byte = ' 我'.encode('utf-8')
  - print(byte,len(byte))

# 9-2-2 編碼與解碼 (ch9\9-2-2- 編碼與解碼.py)

綜合上述，撰寫編碼與解碼程式如下。

| 行號 | 範例 ( : ch9\9-2-2- 編碼與解碼 .py) |
|---|---|
| 1 | print(' 我 '.encode('unicode-escape')) |
| 2 | def utf8(data): |
| 3 | data_byte=data.encode('utf-8') |
| 4 | data2=data_byte.decode('utf-8') |
| 5 | print(' 將 ', data, ' 經由 utf-8 編碼後爲 ', data_byte) |
| 6 | print(' 將 ', data_byte, ' 經由 utf-8 解碼後爲 ', data2) |
| 7 | print(data, ' 的長度爲 ', len(data)) |
| 8 | print(data_byte, ' 的長度爲 ', len(data_byte)) |
| 9 | utf8(' 我 ') |
| 10 | utf8("\u6211") |

# 9-2-2　編碼與解碼 (ch9\9-2-2- 編碼與解碼.py)

- 第1 行：將「我」使用函式encode 以「unicode-escape」進行編碼，顯示編碼結果在螢幕上。

- 第2 到8 行：定義函式utf8，輸入Unicode 字元參數data，呼叫data 的函式encode，以utf-8 為編碼代碼，回傳的data 的utf-8 編碼結果指定給data_byte( 第3 行)
- 呼叫data_byte 的函式decode，以utf-8 為解碼代碼，回傳的data_byte 的utf-8 解碼結果指定給data2( 第4 行)。

- 使用函式print 顯示將data 編碼後data_byte 的結果顯示在螢幕上( 第5 行)，顯示將data_byte 解碼後data2 的結果顯示在螢幕上( 第6 行)
- 顯示將data 與data 的長度在螢幕上( 第7 行)
- 顯示將data_byte 與data_byte 的長度在螢幕上( 第8 行)。

- 第9 行：呼叫函式utf8，以「我」為輸入參數。
- 第10 行：呼叫函式utf8，以「\u6211」為輸入參數。

# 9-2-2 編碼與解碼 (ch9\9-2-2- 編碼與解碼.py)

## 執行結果

```
b'\\u6211'
將 我 經由 utf-8 編碼後爲 b'\xe6\x88\x91'
將 b'\xe6\x88\x91' 經由 utf-8 解碼後爲 我
我 的長度爲 1
b'\xe6\x88\x91' 的長度爲 3
將 我 經由 utf-8 編碼後爲 b'\xe6\x88\x91'
將 b'\xe6\x88\x91' 經由 utf-8 解碼後爲 我
我 的長度爲 1
b'\xe6\x88\x91' 的長度爲 3
```

# 9-3 正規表示式(regular expression)

- 使用正規表示式找尋特定字串是否存在，找尋符合條件的字串，進行字串取代，正規表示式常用於字串的分析與擷取
- Python 提供正規表示式的功能，需要匯入模組re。

# 9-3-1模組re

- 模組re 提供的重要函式，如下表。

表 9-3　模組 re 提供的重要函式

| 函式 | 說明 | 範例程式 | 執行結果 |
|---|---|---|---|
| match(pattern, string) | 找出 string 的開頭是否符合 pattern 格式，若找不到回傳 None，若找到則回傳 match 物件。 | import re<br>s='昔人已乘黃鶴去'<br>ans = re.match('昔人',s)<br>if ans:<br>　print(ans.group()) | 昔人 |
| search(pattern, string) | 找出 string 中第一個符合 pattern 格式的字串，若找不到回傳 None，找到則回傳 match 物件。 | import re<br>s='昔人已乘黃鶴去'<br>ans = re.search('黃鶴',s)<br>if ans:<br>　print(ans.group()) | 黃鶴 |
| findall(pattern, string) | 找出 string 中所有符合 pattern 格式的字串，回傳串列。 | import re<br>s='昔人已乘黃鶴去'<br>ans = re.findall('黃鶴',s)<br>if ans:<br>　print(ans) | ['黃鶴'] |

# 9-3-1模組re

| 函式 | 說明 | 範例程式 | 執行結果 |
|---|---|---|---|
| split(pattern, string) | 將 string 使用 pattern 格式進行分割，最後回傳一個串列。 | import re<br>s='昔人已乘黃鶴去'<br>ans = re.split('已',s)<br>print(ans) | ['昔人','乘黃鶴去'] |
| sub(pattern, repl, string) | 將 string 中符合 pattern 格式的字串以字串 repl 取代，最後回傳取代後的字串。 | import re<br>s='昔人已乘黃鶴去'<br>ans = re.sub('去','來',s)<br>print(ans) | 昔人已乘黃鶴來 |

# 9-3-1模組re

- 範例程式如下，使用「.* 黃鶴」找尋符合條件的最長字串。
  - import re
  - s=' 昔人已乘黃鶴去，此地空余黃鶴樓。'
  - ans = re.match('.* 黃鶴',s)
  - if ans:
  - print(ans.group())
  - else:
  - print(" 找不到開頭是「.* 黃鶴」")

- 程式執行結果如下。
  - 昔人已乘黃鶴去，此地空余黃鶴

# 9-3-1模組re(ch9\9-3-1-match-search.py)

| 行號 | 範例 (  ：ch9\9-3-1-match-search.py) | 執行結果 |
|---|---|---|
| 1 | import re | |
| 2 | s=' 昔人已乘黃鶴去，此地空余黃鶴樓。\ | |
| 3 | 黃鶴一去不復返，白雲千載空悠悠。' | |
| 4 | print(s) | |
| 5 | ans = re.match(' 昔人 ',s) | |
| 6 | if ans: | |
| 7 | print(ans.group()) | |
| 8 | else: | |
| 9 | print(" 找不到開頭是「昔人」 ") | |
| 10 | ans = re.match(' 黃鶴 ',s) | |

```
11  if ans:
12      print(ans.group())
13  else:
14      print(" 找不到開頭是「黃鶴」 ")
15  ans = re.match('.* 黃鶴 ',s)
16  if ans:
17      print(ans.group())
18  else:
19      print(" 找不到開頭是「.* 黃鶴」 ")
20  ans = re.search(' 黃鶴 ',s)
21  if ans:
22      print(ans.group())
23  else:
24      print(" 找不到「黃鶴」 ")
25  ans = re.findall(' 黃鶴 ',s)
```

```
昔人已乘黃鶴去，此地空余黃鶴樓。黃鶴一去不復返，白雲千載空悠悠。
昔人
找不到開頭是「黃鶴」
昔人已乘黃鶴去，此地空余黃鶴樓。黃鶴黃鶴
[' 黃鶴 ', ' 黃鶴 ', ' 黃鶴 ']
[' 昔人已乘黃鶴去 ', ' 此地空余黃鶴樓。黃鶴一去不復返 ', ' 白雲千載空悠悠。']
昔人已乘黃鶴去，此地空余黃鶴樓；黃鶴一去不復返，白雲千載空悠悠；
```

```
26  if ans:
27      print(ans)
28  else:
29      print(" 找不到「黃鶴」 ")
30  ans = re.split(' ， ',s)
31  print(ans)
32  ans = re.sub(' ◦ ',' ； ',s)
33  print(ans)
```

# 9-3-1模組re(ch9\9-3-1-match-search.py)

- 第1 行：匯入模組re。
- 第2 到3 行：字串s 為「昔人已乘黃鶴去，此地空余黃鶴樓。黃鶴一去不復返，白雲千載空悠悠。」

- 第4 行：使用函式print 顯示字串s到螢幕上。
- 第5 行：使用模組re 的函式match，找出字串s 的開頭是否是為「昔人」，將結果指定給變數ans。

- 第6 到9 行：ans 為match 物件，若ans 不是None，則使用函式print 將ans 的函式group 結果顯示到螢幕上；否則顯示「找不到開頭是「昔人」」。
- 第10 行：使用模組re 的函式match，找出字串s 的開頭是否是為「黃鶴」，將結果指定給變數ans。

- 第11 到14 行：ans 為match 物件，若ans 不是None，則使用函式print 將ans 的函式group 結果顯示到螢幕上；否則顯示「找不到開頭是「黃鶴」」。
- 第15 行：使用模組re 的函式match，找出字串s 的開頭是否是為「.* 黃鶴」，將結果指定給變數ans。

- 第16 到19 行：ans 為match 物件，若ans 不是None，則使用函式print 將ans 的函式group 結果顯示到螢幕上；否則顯示「找不到開頭是「.* 黃鶴」」。
- 第20 行：使用模組re 的函式search，找出字串s 的第一個字串為「黃鶴」，將結果指定給變數ans。

- 第21 到24 行：ans 為match 物件，若ans 不是None，則使用函式print 將ans 的函式group 結果顯示到螢幕上；否則顯示「找不到「黃鶴」」。
- 第25 行：使用模組re 的函式findall，找出字串s 中所有字串為「黃鶴」，將結果指定給變數ans。

- 第26 到29 行：ans 為串列，若ans 不是空串列，則顯示變數ans 到螢幕上；否則顯示「找不到「黃鶴」」。
- 第30 行：使用模組re 的函式split，找出字串s 中使用「，」進行分割，將結果指定給變數ans。
- 第31 行：顯示變數ans到螢幕上。

- 第32 行：使用模組re 的函式sub，將字串s使用「；」取代「。」，將取代後的字串指定給變數ans。
- 第33 行：顯示變數ans到螢幕上。

# 9-3-2 正規表示式的關鍵字

- 正規表示式義了一些關鍵字，這些關鍵字對字元進行分類，是否為數字、是否為單字、是否為空白字符等。

表 9-4　正規表示式的關鍵字

| 關鍵字 | 說明 | 範例程式 | 範例程式執行結果 |
|---|---|---|---|
| \d | 匹配一個數字。 | import re<br>s='a1 + b_2'<br>print(re.findall('\d',s)) | ['1', '2'] |
| \D | 匹配一個非數字。 | import re<br>s='a1 + b_2'<br>print(re.findall('\D',s)) | ['a', ' ', '+', ' ', 'b', '_'] |
| \s | 匹配一個空白字符。 | import re<br>s='a1 + b_2'<br>print(re.findall('\s',s)) | [' ', ' '] |
| \S | 匹配一個非空白字符。 | import re<br>s='a1 + b_2'<br>print(re.findall('\S',s)) | ['a', '1', '+', 'b', '_', '2'] |

| | | | |
|---|---|---|---|
| \w | 匹配一個英文、數字或底線字元，也可以匹配一個中文字元。 | import re<br>s='a1 + b_2'<br>print(re.findall('\w',s)) | ['a', '1', 'b', '_', '2'] |
| \W | 匹配不是一個英文、數字或底線字元，也可以匹配中文的標點符號。 | import re<br>s='a1 + b_2'<br>print(re.findall('\W',s)) | [' ', '+', ' '] |
| \b | 匹配「\w」與「\W」的邊界。 | import re<br>print(re.findall(r'\b','abcd')) | ['', ''] |
| \B | 匹配不在「\w」與「\W」的邊界。 | import re<br>print(re.findall('\B','abcd')) | ['', '', ''] |

- 「'\b'」有另一種解釋為backspace，若要指定為正規表示式的匹配「\w」與「\W」的邊界，需在前面加上「r」，變成「r'\b'」確定一定是正規表示式的匹配「\w」與「\W」的邊界，而非backspace。

# 9-3-2 正規表示式的關鍵字 ( ch9\9-3-2a- 正規表示式的關鍵字.py)

綜合上述，正規表示式程式如下。

| 行號 | 範例 ( : ch9\9-3-2a- 正規表示式的關鍵字 .py) |
|---|---|
| 1 | import string |
| 2 | import re |
| 3 | pr=string.printable |
| 4 | print(re.findall('\d',pr)) |
| 5 | print(re.findall('\D',pr)) |
| 6 | print(re.findall('\s',pr)) |
| 7 | print(re.findall('\S',pr)) |
| 8 | print(re.findall('\w',pr)) |
| 9 | print(re.findall('\W',pr)) |
| 10 | print(re.findall(r'\b','abcd')) |
| 11 | print(re.findall('\B','abcd')) |

# 程式解說

- 第1 行：匯入模組string。
- 第2 行：匯入模組re。
- 第3 行：設定變數pr 為模組string 的printable，為可列印字元。
- 第4 行：使用模組re 的函式findall，找出變數pr 中所有數字(\d)。

- 第5 行：使用模組re 的函式findall，找出變數pr 中所有非數字(\D)。
- 第6 行：使用模組re 的函式findall，找出變數pr 中所有空白字符(\s)。

- 第7 行：使用模組re 的函式findall，找出變數pr 中所有非空白字元(\S)。
- 第8行：使用模組re的函式findall，找出變數pr中所有英文、數字或底線字元(\w)。

- 第9 行：使用模組re 的函式findall，找出變數pr 中所有非英文、數字或底線字元(\
- W)。
- 第10行：使用模組re的函式findall，找出變數pr中所有「\w」與「\W」的邊界(\b)。
- 第11行：使用模組re的函式findall，找出變數pr中所有非「\w」與「\W」的邊界(\B)。

## 執行結果

```
['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z',
'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X',
'Y', 'Z', '!', '" ', '#', '$', '%', '&', "' ", '(', ')', '*', '+', ',', '-', '.', '/', ':', ';', '<', '=', '>', '?', '@', '[', '\\', ']',
'^', '_', '`', '{', '|', '}', '~', ' ', '\t', '\n', '\r', '\x0b', '\x0c']
[' ', '\t', '\n', '\r', '\x0b', '\x0c']
['0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p',
'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O',
'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', '!', '" ', '#', '$', '%', '&', "' ", '(', ')', '*', '+', ',', '-', '.',
'/', ':', ';', '<', '=', '>', '?', '@', '[', '\\', ']', '^', '_', '`', '{', '|', '}', '~']
['0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p',
'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O',
'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', '_']
['!', '" ', '#', '$', '%', '&', "' ", '(', ')', '*', '+', ',', '-', '.', '/', ':', ';', '<', '=', '>', '?', '@', '[', '\\', ']',
'^', '`', '{', '|', '}', '~', ' ', '\t', '\n', '\r', '\x0b', '\x0c']
['', '']
['', '', '']
```

# 9-3-2 正規表示式的關鍵字

- 正規表示式定義另一些關鍵字，這些關鍵字表示出現的位置、出現的次數與有哪些字元符合條件等。

| 關鍵字 | 說明 | 範例程式 | 執行結果 |
|---|---|---|---|
| ^ | 位置在開頭。 | import re<br>s=' 昔人已乘黃鶴去 '<br>ans = re.findall('^ 昔人 ',s)<br>print(ans) | [' 昔人 '] |
| $ | 位置在結尾。 | import re<br>s=' 昔人已乘黃鶴去 '<br>ans = re.findall(' 黃鶴去 $',s)<br>print(ans) | [' 黃鶴去 '] |
| . | 配對除了 Enter(\n) 以外的字元。 | import re<br>s=' 昔人已乘黃鶴去 '<br>ans = re.findall('. 已乘 ',s)<br>print(ans) | [' 人已乘 '] |
| x\|y | 配對 x 或 y，x 與 y 可以以任何字元取代。 | import re<br>s=' 昔人已乘黃鶴去 '<br>ans = re.findall(' 已 \| 人 ',s)<br>print(ans) | [' 人 ', ' 已 '] |

| 關鍵字 | 說明 | 範例程式 | 執行結果 |
|---|---|---|---|
| [xy] | 配對 x 或 y，x 與 y 可以以任何字元取代，可以不只兩個字元，例如：[xyz]，表示 x、y 或 z。 | import re<br>s='昔人已乘黃鶴去'<br>ans = re.findall('[已人]',s)<br>print(ans) | ['人', '已'] |
| [^xy] | 不是 x 且不是 y，x 與 y 可以以任何字元取代，可以不只兩個字元。 | import re<br>s='昔人已乘黃鶴去'<br>ans = re.findall('[^已人]',s)<br>print(ans) | ['昔', '乘', '黃', '鶴', '去'] |
| x* | 配對零個或多個 x，越多越好，x 可以以任何字元取代。 | import re<br>s='昔人已乘黃鶴去'<br>ans = re.findall('[已人]*',s)<br>print(ans) | ['', '人已', '', '', '', '', ''] |
| x*? | 配對零個或多個 x，越少越好，x 可以以任何字元取代。 | import re<br>s='昔人已乘黃鶴去'<br>ans = re.findall('[已人]*?',s)<br>print(ans) | ['', '', '', '', '', '', '', ''] |

| | | | |
|---|---|---|---|
| x+ | 配對 1 個或多個 x，越多越好，x 可以以任何字元取代。 | import re<br>s=' 昔人已乘黃鶴去 '<br>ans = re.findall(' [ 已人 ]+',s)<br>print(ans) | [' 人已 '] |
| [0-9]+ | 配對 1 個或多個的 0 到 9 的數字，越多越好。 | import re<br>s = '123342  3423434'<br>ans = re.findall('[0-9]+',s)<br>print(ans) | ['123342',<br>'3423434'] |
| [A-Za-z]+ | 配對 1 個或多個的大小寫英文字母，越多越好。 | import re<br>s = 'Time is money'<br>ans = re.findall('[A-Za-z]+',s)<br>print(ans) | ['Time', 'is',<br>'money'] |
| x+? | 配對 1 個或多個 x，越少越好，x 可以以任何字元取代。 | import re<br>s=' 昔人已乘黃鶴去 '<br>ans = re.findall(' [ 已人 ]+?',s)<br>print(ans) | [' 人 ', ' 已 '] |

| 關鍵字 | 說明 | 範例程式 | 執行結果 |
|---|---|---|---|
| x{a} | 配對連續 a 個 x，x 可以以任何字元取代。 | import re<br>s = 'xxxyy'<br>ans = re.findall('x{2}',s)<br>print(ans) | ['xx'] |
| x{a, b} | 配對連續 a 到 b 個 x，越多越好，x 可以以任何字元取代。 | import re<br>s = 'xxxyy'<br>ans = re.findall('x{2,3}',s)<br>print(ans) | ['xxx'] |
| x{a, b}? | 配對連續 a 到 b 個 x，越少越好，x 可以以任何字元取代。 | import re<br>s = 'xxxyy'<br>ans = re.findall('x{2,3}?',s)<br>print(ans) | ['xx'] |

| | | | |
|---|---|---|---|
| left(?=right) | 配對 left，若後面有 right。 | import re<br>s=' 昔人已乘黃鶴去 '<br>ans = re.findall(' 乘 (?= 黃鶴 )',s)<br>print(ans) | [' 乘 '] |
| left(?!right) | 配對 left，若後面沒有 right。 | import re<br>s=' 昔人已乘黃鶴去 '<br>ans = re.findall(' 乘 (?! 黃鶴 )',s)<br>print(ans) | [] |
| (?<=left) right | 配對 right，若 right 之前有 left。 | import re<br>s=' 昔人已乘黃鶴去 '<br>ans = re.findall('(?<= 乘 ) 黃鶴 ',s)<br>print(ans) | [' 黃鶴 '] |
| (?<!left) right | 配對 right，若 right 之前沒有 left。 | import re<br>s=' 昔人已乘黃鶴去 '<br>ans = re.findall('(?<! 乘 ) 黃鶴 ',s)<br>print(ans) | [] |

| 行號 | 範例 ( ：ch9\9-3-2b- 正規表示式的關鍵字 .py) |
|---|---|
| 1 | import re |
| 2 | s=' 昔人已乘黃鶴去，此地空余黃鶴樓。\ |
| 3 | 黃鶴一去不復返，白雲千載空悠悠。' |
| 4 | print(s) |
| 5 | ans = re.findall('^ 昔人 ',s) |
| 6 | print(ans) |
| 7 | ans = re.findall(' 空悠悠。$',s) |
| 8 | print(ans) |
| 9 | ans = re.findall('[ 黃白 ]',s) |
| 10 | print(ans) |
| 11 | ans = re.findall('\W',s) |
| 12 | print(ans) |
| 13 | ans = re.findall(' 黃鶴樓 \W',s) |
| 14 | print(ans) |
| 15 | ans = re.findall(' 黃鶴 .\W',s) |
| 16 | print(ans) |
| 17 | ans = re.findall('[^ 黃鶴 ]',s) |
| 18 | print(ans) |

# 程式解說

- 第1 行：匯入模組re。
- 第2到3行：設定變數s為「昔人已乘黃鶴去，此地空余黃鶴樓。黃鶴一去不復返，白雲千載空悠悠。」
- 第4 行：使用print 印出變數s到螢幕上。

- 第5 行：使用模組re 的函式findall，找出變數s的開頭是否為「昔人」(^ 昔人)。
- 第6 行：使用print 印出變數ans到螢幕上。
- 第7行：使用模組re的函式findall，找出變數s的結尾是否為「空悠悠。」(空悠悠。$)。
- 第8 行：使用print 印出變數ans到螢幕上。

- 第9 行：使用模組re 的函式findall，找出變數s的所有「黃」或「白」([ 黃白])。
- 第10 行：使用print 印出變數ans到螢幕上。
- 第11行：使用模組re的函式findall，找出變數s的所有非中文字的標點符號 (\W)。
- 第12 行：使用print 印出變數ans到螢幕上。

- 第13 行：使用模組re 的函式findall，找出變數s 的所有「黃鶴樓」後面串接一個非中文字的字串( 黃鶴樓\W)。
- 第14 行：使用print 印出變數ans到螢幕上。
- 第15行：使用模組re的函式findall，找出變數s的所有「黃鶴」後面串接一個字元，再串接一個非中文字的字串( 黃鶴.\W)。

- 第16 行：使用print 印出變數ans到螢幕上。
- 第17行：使用模組re的函式findall，找出變數s的所有不是「黃」且也不是「鶴」的字 ([^ 黃鶴])。
- 第18 行：使用print 印出變數ans到螢幕上。

# 9-3-2　正規表示式的關鍵字 (ch9\9-3-2c- 正規表示式的關鍵字.py)

進階正規表示式程式，如下。

| 行號 | 範例 (  ：ch9\9-3-2c- 正規表示式的關鍵字 .py) |
|---|---|
| 1 | import re |
| 2 | s=' 昔人已乘黃鶴去，此地空余黃鶴樓。\ |
| 3 | 黃鶴一去不復返，白雲千載空悠悠。' |
| 4 | print(s) |
| 5 | ans = re.findall('[ 一去 ]?', s) |
| 6 | print('[ 一去 ]?', ans) |
| 7 | ans = re.findall('[ 一去 ]*', s) |
| 8 | print('[ 一去 ]*', ans) |
| 9 | ans = re.findall('[ 一去 ]*?', s) |
| 10 | print('[ 一去 ]*?', ans) |
| 11 | ans = re.findall('[ 黃鶴樓 ]+',s) |
| 12 | print('[ 黃鶴樓 ]+', ans) |
| 13 | ans = re.findall('[ 黃鶴樓 ]+?',s) |
| 14 | print('[ 黃鶴樓 ]+?', ans) |

# 9-3-2　正規表示式的關鍵字 (ch9\9-3-2c- 正規表示式的關鍵字.py)

```
15  ans = re.findall('[ 黃鶴樓 ]{2}',s)
16  print('[ 黃鶴樓 ]{2}', ans)
17  ans = re.findall('[ 黃鶴樓 ]{1,2}',s)
18  print('[ 黃鶴樓 ]{1,2}', ans)
19  ans = re.findall('[ 黃鶴樓 ]{1,2}?',s)
20  print('[ 黃鶴樓 ]{1,2}?', ans)
21  ans = re.findall(' 黃鶴 (?= 樓 )',s)
22  print(' 黃鶴 (?= 樓 )', ans)
23  ans = re.findall(' 黃鶴 (?! 樓 )',s)
24  print('[ 黃鶴 (?! 樓 )', ans)
25  ans = re.findall('(?<= 黃鶴 ) 樓 ',s)
26  print('(?<= 黃鶴 ) 樓 ', ans)
27  ans = re.findall('(?<! 黃鶴 ) 樓 ',s)
28  print('(?<! 黃鶴 ) 樓 *', ans)
```

# 程式解說

- 第1 行：匯入模組re。
- 第2到3行：設定變數s為「昔人已乘黃鶴去，此地空余黃鶴樓。黃鶴一去不復返，白雲千載空悠悠。」
- 第4 行：使用函式print 印出變數s到螢幕上。

- 第5行：使用模組re的函式findall，找出變數s中0個或1個「一」或「去」([一去]?)。
- 第6 行：使用函式print 印出變數ans到螢幕上。
- 第7 行：使用模組re 的函式findall，找出變數s 中0 個或多個「一」或「去」，越多越好([ 一去]*)。

- 第8 行：使用函式print 印出變數ans到螢幕上。
- 第9 行：使用模組re 的函式findall，找出變數s 中0 個或多個「一」或「去」，越少越好([ 一去]*?)。

- 第10 行：使用函式print 印出變數ans到螢幕上。
- 第11行：使用模組re的函式findall，找出變數s中1個或多個「黃」、「鶴」或「樓」，越多越好([ 黃鶴樓]+)。
- 第12 行：使用函式print 印出變數ans到螢幕上。
- 第13行：使用模組re的函式findall，找出變數s中1個或多個「黃」、「鶴」或「樓」，越少越好([ 黃鶴樓]+?)。。

- 第14 行：使用函式print 印出變數ans到螢幕上。
- 第15行：使用模組re的函式findall，找出變數s中「黃」、「鶴」或「樓」連續兩個([黃鶴樓]{2})。

- 第16 行：使用函式print 印出變數ans到螢幕上。
- 第17 行：使用模組re 的函式findall，找出變數s 中「黃」、「鶴」或「樓」連續一個到兩個，越多越好([ 黃鶴樓]{1,2})。
- 第18 行：使用函式print 印出變數ans到螢幕上。

## 執行結果

```
昔人已乘黃鶴去，此地空余黃鶴樓。黃鶴一去不復返，白雲千載空悠悠。
[一去]? ['', '', '', '', '', '', '去', '', '', '', '', '', '', '', '', '', '', '', '一', '去', '', '', '', '', '', '', '', '', '', '', '']
[一去]* ['', '', '', '', '', '', '去', '', '', '', '', '', '', '', '', '', '', '一去', '', '', '', '', '', '', '', '', '', '', '']
[一去]*? ['', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '']
[黃鶴樓]+ ['黃鶴', '黃鶴樓', '黃鶴']
[黃鶴樓]+? ['黃', '鶴', '黃', '鶴', '樓', '黃', '鶴']
[黃鶴樓]{2} ['黃鶴', '黃鶴', '黃鶴']
[黃鶴樓]{1,2} ['黃鶴', '黃鶴', '樓', '黃鶴']
[黃鶴樓]{1,2}? ['黃', '鶴', '黃', '鶴', '樓', '黃', '鶴']
黃鶴(?=樓) ['黃鶴']
黃鶴(?!樓) ['黃鶴', '黃鶴']
(?<=黃鶴)樓 ['樓']
(?<!黃鶴)樓 []
```